

Dev.to Articles ETL Pipeline Documentation

Project Overview

This project implements an end-to-end ETL pipeline using Python, Prefect, PostgreSQL, SQLAlchemy, Pandas, and HTTPX. The pipeline extracts article data from the Dev.to API, transforms it into a structured format, loads it into PostgreSQL, and performs incremental loading using a watermark-based approach.

Architecture

Dev.to API -> Prefect Flow -> Extract -> Transform -> Load -> SQLAlchemy ORM -> PostgreSQL -> ETL Metadata Table

Project Structure

database.py, models.py, create_tables.py, etlflow.py

database.py

Creates the SQLAlchemy engine, session factory, and Base object. The database connection string is read from the DATABASE_URL environment variable.

models.py

Defines the Articles and ETLMetadata SQLAlchemy ORM models. Articles stores article data; ETLMetadata stores the incremental loading watermark.

create_tables.py

Creates database tables using Base.metadata.create_all(engine). Run this once before executing the ETL flow.

ETL Flow

1. Extract data from the Dev.to API.
2. Transform JSON into a Pandas DataFrame.
3. Convert published_at into timezone-aware UTC timestamps.
4. Read previous watermark from ETLMetadata.
5. Filter only records newer than the watermark.
6. Load records into PostgreSQL using SQLAlchemy merge().
7. Update the watermark with the maximum published_at value loaded.

Incremental Loading

The ETLMetadata table stores the last successfully processed article timestamp. During subsequent executions, only newer articles are loaded.

Database Tables

articles: stores article data and load timestamps. etl_metadata: stores pipeline_name and last_run watermark.

Execution Order

1. Set the DATABASE_URL environment variable. 2. Run create_tables.py once. 3. Run etlflow.py for every ETL execution.

Technologies Used

Python, Prefect, SQLAlchemy, PostgreSQL, Pandas, HTTPX.

Future Enhancements

Dockerization, Prefect deployments and scheduling, monitoring dashboards, PostgreSQL ON CONFLICT upserts, audit metrics and reporting.